

Professores:

Celso Giusti

Daniel Manoel Filho

Felipe Augusto Stevanim Gregate

Marlon Palata Fanger Rodrigues

DOM AVANÇADO



Revisão Rápida: A Árvore DOM

Na Aula 05, aprendemos que o DOM organiza o HTML como uma **árvore de pai e filho**. Até agora, usamos seletores para "pular" direto para um elemento pelo ID ou classe.

Agora vamos dar um passo além: **navegar pela árvore a partir de um elemento já encontrado**, como subir para o pai, descer para os filhos ou andar para o lado (irmãos).

```
document
├── <body>
│   ├── <section class="lista-noticias"> ← PAI
│   │   ├── <article class="card"> ← FILHO 1 (firstElementChild)
│   │   ├── <article class="card"> ← FILHO 2
│   │   └── <article class="card"> ← FILHO 3 (lastElementChild)
```

Lembre-se: `querySelector` acha o elemento. A navegação na árvore parte de um elemento já encontrado.

Navegação: parentElement

A propriedade **parentElement** retorna o **elemento pai** direto de um nó. É usada quando você encontrou um filho e precisa acessar ou modificar o container que o envolve.

Onde usar: Remover um card inteiro ao clicar no botão "Fechar" que está dentro dele; destacar a seção que contém o elemento clicado.

```
const btnFechar = document.querySelector('.btn-fechar');
```

```
// A partir do botão, subimos para o card que o contém
```

```
const card = btnFechar.parentElement;
```

```
// Agora podemos manipular o card inteiro
```

```
card.style.display = 'none'; // Esconde o card pai
```

Atenção: parentElement sobe **um nível** apenas. Para subir dois níveis use **elemento.parentElement.parentElement**.

Navegação: children e childNodes

Enquanto **parentElement** sobe na árvore, **children** desce — retorna todos os filhos **diretos** de um elemento em uma **HTMLCollection**.

```
const secao = document.querySelector('.lista-noticias');  
// Retorna todos os <article> filhos diretos  
const filhos = secao.children;  
console.log(filhos.length); // Quantidade de cards  
console.log(filhos[0]); // Primeiro card
```

Propriedade	Retorna	Inclui texto/comentários?
children	HTMLCollection	✗ Não — só elementos HTML
childNodes	NodeList	✓ Sim — inclui textos e comentários

Lembre-se: Na prática, use sempre **children**. O **childNodes** inclui "nós de texto" invisíveis (espaços e quebras de linha do HTML) que causam confusão.

Navegação: firstElementChild e lastElementChild

Atalhos práticos para acessar **o primeiro e o último filho** de um elemento diretamente, sem precisar usar índices.

```
const lista = document.querySelector('.lista-noticias');  
// Primeiro card da lista  
const primeiroCard = lista.firstElementChild;  
primeiroCard.classList.add('noticia-destaque');  
// Último card da lista  
const ultimoCard = lista.lastElementChild;  
ultimoCard.classList.add('noticia-nova');
```

Importante: Se o elemento não tiver filhos, ambos retornam **null**. Sempre verifique antes de manipular:

```
if (lista.firstElementChild) { ... }
```


Navegação: `nextElementSibling` e `previousElementSibling`

Permitem **andar para o lado** na árvore — acessar o irmão seguinte ou anterior de um elemento.

```
const cardAtivo = document.querySelector('.card-ativo');
```

```
// Próximo irmão (card seguinte na lista)
```

```
const proximo = cardAtivo.nextElementSibling;
```

```
// Irmão anterior (card anterior na lista)
```

```
const anterior = cardAtivo.previousElementSibling;
```

```
if (proximo) {  
    proximo.classList.add('proximo-destaque');  
}
```

Onde usar: Navegação entre abas, carrosséis de imagens, listas de passos (step-by-step).

Criando Elementos: createElement

O **createElement** cria um **novo elemento HTML na memória**, sem inseri-lo na página ainda. É o primeiro passo para adicionar conteúdo dinamicamente — como renderizar cards vindos de uma API.

// 1. CRIAR o elemento (ainda não está na página)

```
const novoCard = document.createElement('article');
```

// 2. CONFIGURAR o elemento

```
novoCard.classList.add('card')
```

```
novoCard.innerHTML = `
```

```
  <h2>Nova Notícia</h2>
```

```
  <p>Conteúdo dinâmico gerado pelo JavaScript.</p>
```

```
  <button class="btn-leitura">Leia Mais</button>`
```

Atenção: Criar o elemento com **createElement** não o insere na página. Para isso, usamos **appendChild**, **prepend** ou **insertAdjacentHTML** (próximos slides).

Inserindo Elementos: appendChild e prepend

Após criar o elemento, precisamos **inserir-lo na árvore DOM**.

```
const lista =  
document.querySelector('.lista-noticias');  
const novoCard = document.createElement('ar  
novoCard.classList.add('card');  
novoCard.textContent = 'Notícia gerada  
dinamicamente!';  
// appendChild → insere como ÚLTIMO filho  
lista.appendChild(novoCard);  
  
// prepend → insere como PRIMEIRO filho  
lista.prepend(novoCard);
```

Método	Onde insere
appendChild(el)	Como último filho do pai
prepend(el)	Como primeiro filho do pai
append(el ou texto)	Como último filho (aceita texto direto também)

Inserindo Elementos: insertAdjacentHTML

Método poderoso que insere HTML em **4 posições precisas** ao redor de um elemento de referência, sem substituir o conteúdo existente.

```
const card = document.querySelector('.card-destaque');  
  
// ANTES do elemento (fora dele, como irmão anterior)  
card.insertAdjacentHTML('beforebegin', '<div class="aviso">Em Alta!</div>');  
  
// INÍCIO do conteúdo interno (primeiro filho)  
card.insertAdjacentHTML('afterbegin', '<span class="tag">Novo</span>');  
  
// FIM do conteúdo interno (último filho)  
card.insertAdjacentHTML('beforeend', '<button>Compartilhar</button>');  
  
// DEPOIS do elemento (fora dele, como irmão seguinte)  
card.insertAdjacentHTML('afterend', '<hr class="divisor">');
```

Clonando Elementos: cloneNode

O **cloneNode** cria uma **cópia exata** de um elemento existente. Útil quando precisamos duplicar um card ou template sem reescrever todo o HTML.

```
const cardOriginal = document.querySelector('.card:first-child');
const lista = document.querySelector('.lista-noticias');
// true → clona o elemento E todo o seu conteúdo interno
const copia = cardOriginal.cloneNode(true);
// false → clona APENAS o elemento, sem os filhos
const copiaVazia = cardOriginal.cloneNode(false);
// Inserindo a cópia no final da lista
lista.appendChild(copia);
```

Onde usar: Duplicar templates de cards, criar linhas em tabelas dinâmicas, replicar componentes de formulário.

Atenção: Se o elemento original tiver um id, a cópia terá o **mesmo ID**, o que quebra a unicidade. Lembre-se de alterar o ID após clonar.

Removendo Elementos: `remove()` e `removeChild()`

Para **deletar um elemento** da página dinamicamente. Dois métodos: o moderno e o antigo.

// MODERNO: O elemento se remove sozinho

```
const cardAntigo = document.querySelector('.card-expirado');
```

```
cardAntigo.remove();
```

// ANTIGO: O pai remove o filho

```
const lista = document.querySelector('.lista-noticias');
```

```
const cardAlvo = document.querySelector('#card-3');
```

```
lista.removeChild(cardAlvo);
```

Método	Como funciona	Quando usar
elemento.remove()	O elemento se remove	Padrão moderno – use sempre
pai.removeChild(filho)	O pai remove um filho	Compatibilidade com navegadores antigos

Onde usar: Botão "Fechar" em notificações, remover item do carrinho, apagar linha de uma tabela.

Eventos Avançados:

event.preventDefault()

Por padrão, certos elementos HTML têm **comportamentos automáticos** que podem atrapalhar o JavaScript:

- Um **<form>** ao ser enviado **recarrega a página**.
- Um **** **pula para o topo** da página.

O **preventDefault()** **cancela esse comportamento padrão** sem parar o evento.

```
const formulario = document.querySelector('#form-contato');  
formulario.addEventListener('submit', (event) => {  
  // IMPEDE o recarregamento da página  
  event.preventDefault();  
  // Agora podemos processar os dados com JS  
  const nome = document.querySelector('#nome').value;  
  console.log('Formulário enviado por: ' + nome);  
});
```

Importante: Sem `event.preventDefault()` em formulários, a página recarrega e todos os dados JS são perdidos. Este é um dos erros mais comuns de iniciantes.

Bubbling e event.stopPropagation()

Event Bubbling (Propagação): Quando um evento ocorre em um elemento filho, ele **sobe automaticamente** para os elementos pai, como uma bolha subindo na água.

```
<section id="secao">      <!-- 3º a ser acionado -->
  <div id="card">          <!-- 2º a ser acionado -->
    <button id="btn">Clique</button> <!-- 1º a ser acionado -->
  </div>
</section>
```

```
const btn = document.querySelector('#btn');
const card = document.querySelector('#card');
btn.addEventListener('click', (event) => {
  console.log('Cliquei no BOTÃO');
  // Impede que o evento suba para o card e a seção
  event.stopPropagation();
});
card.addEventListener('click', () => {
  console.log('Cliquei no CARD'); // Não será executado
});
```

Onde usar: Menus dropdown — você quer que o clique no item do menu feche o menu, mas não acione o clique no fundo da página.

Delegação de Eventos

Em vez de adicionar um **addEventListener** em cada elemento filho (ineficiente), adicionamos **um único ouvinte no pai** e usamos **event.target** para identificar quem foi clicado.

```
const lista = document.querySelector('.lista-noticias');
// UM ouvinte no pai vigia TODOS os filhos
lista.addEventListener('click', (event) => {
  const clicado = event.target;

  // Verifica se o clique foi num botão de leitura
  if (clicado.classList.contains('btn-leitura')) {
    const card = clicado.parentElement;
    card.classList.toggle('card-expandido');
    console.log('Card expandido: ' + card.querySelector('h2').textContent);
  }
});
```

Vantagem real: Se novos cards forem criados dinamicamente (como via API), eles **automaticamente** respondem ao clique — sem precisar adicionar novos listeners.

removeEventListener

Remove um ouvinte de evento previamente adicionado. Importante para **evitar vazamentos de memória** e comportamentos duplicados.

```
// A função precisa ter NOME para poder ser removida depois
```

```
function aoClicar() {
```

```
  console.log('Botão clicado!');
```

```
}
```

```
const btn = document.querySelector('#btn-enviar');
```

```
// Adiciona o ouvinte
```

```
btn.addEventListener('click', aoClicar);
```

```
// Remove o ouvinte (o botão para de responder)
```

```
btn.removeEventListener('click', aoClicar);
```

Atenção: Funções anônimas (**function() {}** ou **() => {}**) **não podem ser removidas** com **removeEventListener**. A função precisa ter um nome e ser a mesma referência.

Onde usar: Desativar um botão após o primeiro clique (evitar duplo envio de formulário), limpar eventos ao trocar de "tela" em aplicações de página única (SPA).

- ✓ **parentElement** = Acessa o elemento pai direto na árvore DOM.
- ✓ **children** = Lista todos os filhos diretos (somente elementos HTML).
- ✓ **firstElementChild / lastElementChild** = Atalho para o primeiro e último filho.
- ✓ **nextElementSibling / previousElementSibling** = Navega entre irmãos na árvore.
- ✓ **createElement + appendChild/prepend** = Cria e insere elementos dinamicamente.
- ✓ **insertAdjacentHTML** = Insere HTML em 4 posições precisas ao redor de um elemento.
- ✓ **cloneNode(true/false)** = Clona um elemento com ou sem seus filhos.
- ✓ **remove() / removeChild()** = Remove elementos da página dinamicamente.
- ✓ **event.preventDefault()** = Cancela o comportamento padrão do navegador (ex: recarregar form).
- ✓ **Bubbling + stopPropagation()** = Eventos sobem na árvore; stopPropagation interrompe a subida.
- ✓ **Delegação de Eventos** = Um ouvinte no pai monitora todos os filhos — mais eficiente.
- ✓ **removeEventListener** = Remove um ouvinte; a função precisa ter nome para funcionar.

Portal EcoCycle

1

Navegação na Árvore

Contexto: Portal EcoCycle (Notícias de Ecologia)

No seu arquivo **eco.js**, realize as seguintes tarefas usando **apenas navegação na árvore** (sem usar **querySelector** com ID ou classe após a seleção inicial):

1. Selecione a seção **.lista-noticias** e exiba no console **quantos cards** ela possui (use **.children.length**).
2. A partir da seção, acesse o **primeiro card** e adicione a classe **noticia-destaque**.
3. A partir do primeiro card, acesse o **próximo irmão** e altere o título (**h2**) para **"Notícia em Alta"**.

Dica: Você vai encadear propriedades:

lista.firstElementChild.nextElementSibling.querySelector('h2')

Portal EcoCycle

2

Criando Elementos Dinamicamente

Contexto: Portal EcoCycle (Notícias de Ecologia)

Crie uma função chamada **adicionarNoticia(titulo, descricao)** que:

1. Crie um novo **<article>** com a classe **card**.
2. Defina o **innerHTML** do card com um **<h2>** (título), um **<p>** (descrição) e um **<button class="btn-leitura">Leia Mais</button>**.
3. Insira o novo card como **primeiro elemento** da **.lista-noticias** (use **prepend**).

Teste chamando a função no console:

```
adicionarNoticia("Reciclagem Bate Recorde", "Brasil atinge 80% de reciclagem de alumínio em 2025.");
```

Portal EcoCycle

3

Removendo e Clonando

Contexto: Portal EcoCycle (Notícias de Ecologia)

1. **Remover:** Adicione um evento de **click** ao botão de cada card. Ao clicar, o card inteiro deve ser removido da página. Use **event.target.parentElement** para acessar o card a partir do botão e **.remove()** para deletá-lo.
2. **Clonar:** Crie um botão fixo na página com o texto "Duplicar Primeira Notícia". Ao clicar, ele deve clonar o **firstElementChild** da lista e inseri-lo no final com **appendChild**.

Atenção: Após clonar, altere o título do clone para "[CÓPIA] " + **tituloOriginal** para diferenciá-lo visualmente.

Portal EcoCycle

4

Formulário Sem Recarregar

Contexto: Portal EcoCycle (Notícias de Ecologia)

No seu **index.html**, adicione uma seção de **"Sugerir Notícia"** com um formulário contendo:

- Um **<input type="text" id="titulo-sugestao">** para o título.
- Um **<textarea id="desc-sugestao">** para a descrição.
- Um **<button type="submit">Enviar Sugestão</button>**.

No JavaScript:

1. Capture o evento **submit** do formulário.
2. Use **event.preventDefault()** para impedir o recarregamento.
3. Chame a função **adicionarNoticia()** da Atividade 2 com os valores dos campos.
4. Limpe os campos após o envio (**campo.value = ""**).

Portal EcoCycle

5

Delegação de Eventos

Contexto: Portal EcoCycle (Notícias de Ecologia)

Aplique **delegação de eventos** na **.lista-noticias**:

1. Remova todos os **addEventListener** individuais que você colocou nos botões **.btn-leitura** nas aulas anteriores.
2. Adicione **um único addEventListener** de **click** na **.lista-noticias**.
3. Dentro do ouvinte, use **event.target** para verificar:
 - Se o clique foi num **.btn-leitura**: alterne a classe **card-expandido** no card pai.
 - Se o clique foi num **.btn-fechar** (se existir): remova o card inteiro.
4. Adicione um novo card dinamicamente via console e confirme que o novo card **também responde** ao clique sem precisar de um novo **addEventListener**.

MOZILLA. **Node.parentElement**. MDN Web Docs. Disponível em:

<https://developer.mozilla.org/pt-BR/docs/Web/API/Node/parentElement>. Acesso em: abr. 2026.

MOZILLA. **Document.createElement()**. MDN Web Docs. Disponível em:

<https://developer.mozilla.org/pt-BR/docs/Web/API/Document/createElement>. Acesso em: abr. 2026.

MOZILLA. **Element.insertAdjacentHTML()**. MDN Web Docs. Disponível em:

<https://developer.mozilla.org/pt-BR/docs/Web/API/Element/insertAdjacentHTML>. Acesso em: abr. 2026.

MOZILLA. **Event.preventDefault()**. MDN Web Docs. Disponível em:

<https://developer.mozilla.org/pt-BR/docs/Web/API/Event/preventDefault>. Acesso em: abr. 2026.

MOZILLA. **Event bubbling**. MDN Web Docs. Disponível em:

https://developer.mozilla.org/pt-BR/docs/Learn/JavaScript/Building_blocks/Event_bubbling. Acesso em: abr. 2026.

JAVASCRIPT.INFO. **Bubbling and capturing**. Disponível em:

<https://javascript.info/bubbling-and-capturing>. Acesso em: abr. 2026.

SENAI-SP. **Desenvolvimento de Sistemas: Front-end**. São Paulo: Editora SENAI, 2023.



ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

Telefone

(11) 2396-1999

Instagram

@senai.itu

Facebook

/senaisp.itu

Site

<https://sp.senai.br/unidade/itu/>